

OTIMIZAÇÃO DE HIPERPARÂMETROS

DCE770 - Heurísticas e Metaheurísticas

Atualizado em: 16 de novembro de 2025

Iago Carvalho

Departamento de Ciência da Computação



PARÂMETROS DE ALGORITMOS

Em todo nosso curso vimos heurísticas e metaheurísticas que possuem diversos parâmetros

- Estes parâmetros devem ser ajustados

Um bom conjunto de parâmetros faz toda a diferença na execução dos algoritmos

- A diferença de um mesmo algoritmo com bons e maus parâmetros é gigante

Uma heurística deve ser comparada com outra em seu *ápice*

- Melhor conjunto de parâmetros possível

Este é um problema combinatório conhecido como **problema de configuração de algoritmos**

O objetivo dele não é simplesmente ajustar os valores numéricos de algoritmos. Ele também se preocupa em projetar o melhor algoritmo possível a partir de um espaço de configuração de componentes

Existem dois tipos possíveis de decisão

1. Decisões categóricas
2. Decisões condicionais

DESAFIOS DO PROBLEMA DE CONFIGURAÇÃO DE ALGORITMOS

O problema de configuração de algoritmos, em si, é polinomial

$$\prod_{i=1}^k p_i$$

onde k é o número de diferentes parâmetros de um algoritmo e p_i é o número de possíveis valores para o i -ésimo parâmetro

Seja $n = \max(p_i | i \in \{1, \dots, k\})$. Temos que

$$\prod_{i=1}^k p_i = \mathcal{O}(kn)$$

DESAFIOS DO PROBLEMA DE CONFIGURAÇÃO DE ALGORITMOS

Suponha que $n = \max(p_i | i \in \{1, \dots, k\}) = 10$. Além disso, suponha que o algoritmo tenha $k = 4$ parâmetros

Além disso, suponha que a heurística leve 10 segundos para rodar

- Você quer fazer a otimização de parâmetros utilizando 50 instâncias e 30 execuções repetidas para garantir robustez estatística

$$\underbrace{10}_{\text{segundos}} \times \underbrace{10 \times 4}_{n \ k} \times \underbrace{50}_{\text{instâncias}} \times \underbrace{30}_{\text{repetições}} = 600.000 \text{ s} = 166,6 \text{ horas}$$

Esse tipo de abordagem por força-bruta (*grid search*) torna-se inviável a depender do conjunto de parâmetros a serem avaliados

MÉTODOS DE CONFIGURAÇÃO DE PARÂMETROS

Então, como podemos resolver o problema de configuração de algoritmos?

1. Chute?
2. Copiar do coleguinha?
3. Busca aleatória?
4. **Estatística!**

Deve-se montar um delineamento experimental robusto o suficiente capaz de resolver o problema de configuração de algoritmos

PORQUÊ A AVALIAÇÃO ESTATÍSTICA É IMPORTANTE

Heurísticas são algoritmos estocásticos

- Totalmente dependente de fatores aleatórios

Pode existir **variação entre instâncias**

- Um conjunto de parâmetros funciona muito bem em uma classe de instâncias
- Funciona muito mal em outras classes

Também pode existir **variação estocástica**

- Algoritmo pode variar muito em diferentes execuções
- Uma execução é muito boa e outra é muito ruim
- Baixa robustez

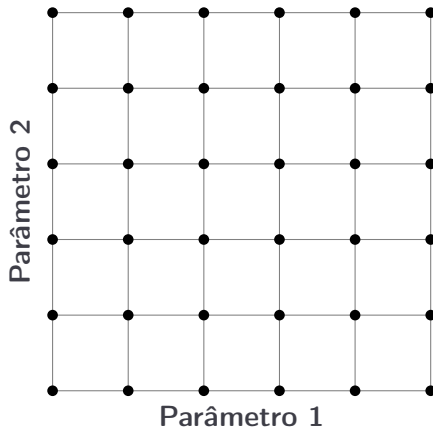
Vamos falar sobre quatro diferentes metodologias

1. Grid search (força-bruta)
2. Random search (busca aleatória)
3. Métodos de corrida
4. Otimização baseada em modelo sequencial
 - 4.1 Otimização bayesiana
 - 4.2 Estimadores de Árvore de Parzen

GRID SEARCH

Fazemos uma discretização nos possíveis valores de cada parâmetro

- Avaliamos todas as intersecções entre esses possíveis valores



RANDOM SEARCH

Chuta-se diversos conjuntos de valores para cada parâmetro

- Chutes são realizados em conjunto
- Quanto mais *chutes*, melhor a chance de obter um bom conjunto de parâmetros

Por exemplo, suponha que um algoritmo tenha três parâmetros A, B e C reais no intervalo $[0, 1]$. Podemos chutar os parâmetros

- $(0.1, 0.3, 0.9)$
- $(0.3, 1.0, 0.7)$
- $(0.9, 0.1, 0.9)$
- $(0.2, 0.4, 0.0)$
- ...

MÉTODO DE CORRIDA

Métodos de corrida (*racing*) são extremamente úteis para diminuir o tempo utilizado em delineamentos *grid search*

- O mais comum é o *F-Race*
 - Baseado no Teste de Friedman

Este é um método iterativo que utiliza testes estatísticos (no caso do *F-Race*, o teste de Friedman) para *deletar* pontos ruins do *grid search*

- Testes estatísticos são utilizados de maneira intercalar as avaliações de instâncias

MÉTODO DE CORRIDA

Ele se inicia com um conjunto nk de pontos

- Cada ponto pode ser entendido como um conjunto de configurações candidatas ou um conjunto de valores para cada parâmetro

O método de corrida realiza os seguintes passos de forma iterativa

1. Executa todas as nk configurações em uma única instância (ou poucas instâncias)
2. Realiza um teste estatístico para comparar as nk configurações
3. Realiza uma eliminação estatística
 - Configurações estatisticamente inferiores as outras são eliminadas

Cada iteração avalia diferentes instâncias até que tenhamos somente uma configuração restante

Não é mais baseado em *grid search*

- Este é considerado uma busca inteligente adaptativa
- Ele *aprende* sobre o espaço de parâmetros conforme é executado

Este algoritmo também é iterativo e é composto de cinco passos

OTIMIZAÇÃO BASEADA EM MODELOS SEQUENCIAIS

1. **Amostragem inicial:** Avalia um pequeno número de configurações (10 ou menos) utilizando *random search*
2. **Construção de modelo substituto:** Ajuste de um modelo substituto aos pares (configuração, custo) observados. Este modelo é uma aproximação estatística e fácil de otimizar da função $f(\text{parâmetros}) \rightarrow \text{custo}$
3. **Otimização do modelo substituto:** Otimiza-se este modelo para encontrar um novo conjunto de parâmetros
4. **Avaliação:** Avalia-se as instâncias utilizando este novo conjunto de parâmetros encontrado
5. **Atualização:** Incrementa o conjunto de configurações iniciais com o novo conjunto de parâmetros encontrados e retorna ao passo 2

O modelo substituto pode ser construído e otimizado de diversas formas. As mais populares são

- Otimização bayesiana
- Estimadores de Árvore de Parzen

Em geral, otimização bayesiana é extremamente elegante e funcional, mas muito cara

- $\mathcal{O}(n^3)$, onde n é o número de configurações

Já métodos baseados em estimadores de árvore de parzen são mais eficientes ($\mathcal{O}(n)$) e conseguem lidar com parâmetros condicionais

IMPLEMENTAÇÕES DOS MÉTODOS DE CONFIGURAÇÃO DE PARÂMETROS

Existem duas bibliotecas extremamente famosas que realizam a otimização de parâmetros de algoritmos

- *irace*
- *optuna*

O *irace* é uma biblioteca disponível na linguagem R que implementa o método de corrida

Já o *optuna* é uma biblioteca do Python que implementa o método de otimização baseada em modelo sequencial

- Utiliza primariamente estimadores de árvore de parzen

O *irace* é a biblioteca mais tradicional para otimizar parâmetros de algoritmos de otimização

- Utilizado tanto para algoritmos exatos como heurísticos

O *irace* pode realizar a otimização de parâmetros de algoritmos desenvolvidos em qualquer linguagem de programação

É necessário definir o *grid* completo de parâmetros

- Conjunto de parâmetros e possíveis valores para cada parâmetro

Já o *optuna* é extremamente utilizado pela comunidade de aprendizado de máquina

O *optuna* foi desenvolvido em *Python* e é utilizado primariamente para otimizar os parâmetros de algoritmos também desenvolvidos em *Python*

- Um pouco difícil de adaptar a outras linguagens de programação

O *optuna* não utiliza um *grid*. Somente é necessário definir o valor mínimo e máximo de cada parâmetro

O *optuna* utiliza somente uma instância de teste

PRÓXIMA AULA:
APRESENTAÇÃO FINAL DOS
TRABALHOS